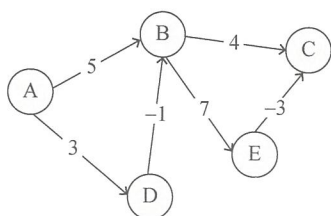


提高组 CSP-S 2024 初赛模拟卷 6

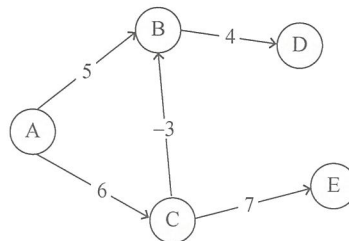
一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 在 Linux 系统终端中，cp 命令的作用是（ ）。
A. 切换工作目录
B. 远程复制文件
C. 复制文件或目录
D. 在文件与文件之间建立 couple 关系
- 8 位无符号二进制数 10110001 和 10110111 相加，运算结果为无符号二进制数（ ）。
A. 101101000 B. 01101000 C. 101001000 D. 01001000
- 给出一个数字序列 1 5 2 7 3 6 4，现希望通过每次交换其中两个数字，使得序列变成数字序列 5 7 4 2 6 3 1，那么最少需要交换多少次？（ ）
A. 3 B. 4 C. 5 D. 6
- 快速排序是一种关键字比较排序算法，需要比较和交换这两个环节。若选择的比较基准值总是当前子序列的第一个数字，那么对于数字序列 1 2 3 4 5 6 7 8，使用快速排序让其从小到大排序将需要多少次交换？（ ）
A. 4 B. 36 C. 72 D. 0
- 在求解最短路径问题的算法中，Dijkstra 算法在处理没有负权边的图时才能保证正确。但在有些出现了负权边的图中，该算法也能得到正确答案。下面的选项起始点都为点 A，请问哪张图使用 Dijkstra 算法可以正确地求出起始点到所有其他点的最短距离？（ ）

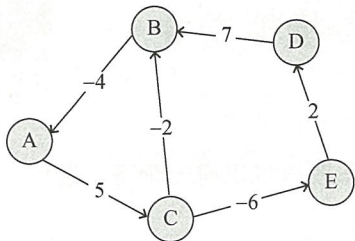
A.



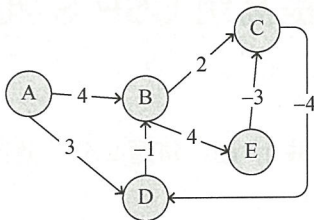
B.



C.

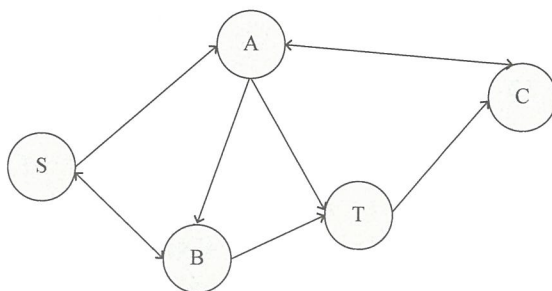


D.



6. 现在有一棵树用 DFS 遍历得到的点为 ABCD, 用 BFS 遍历得到的点也为 ABCD, 同一层的结点从左到右顺序遍历。请问在这棵树有可能的形态上得到的平均深度是多少 (不同的形态为至少有两个不同的点在不同深度, 根结点的深度为 1)? ()
- A. 1.5 B. 3.0 C. 2.0 D. 1.0
7. 现给出一个深度为 n ($n \geq 2$) 的完美二叉树, 请问自下往上、自右往左删掉多少个点, 一定可以得到深度为 $n-1$ 的完全二叉树? ()
- A. $2^{n-1} + 2^{n-2}$ B. 2^{n-2} C. $2n$ D. 2^{n-1}
8. 给出数字序列 1 2 3 4 5 6, 对这些数字进行排列, 假设分别用 $x_1 \sim x_6$ 表示排列后的数字。对于每个数字 x_i 保证 $x_{i+1}、x_{i+2} \leq x_i + 2$ 。请问这样的排列方式有多少种? ()
- A. 50 B. 60 C. 70 D. 80
9. 食堂有 8 个档口, 编号为 1~8, 每个档口有一个美味值。同学们常常会因为不知道吃什么而摇摆不定。小豪想出了一种非常棒的选择方法。现在有 8 个档口可供选择, 使用骰子, 若摇出了 1~2, 则从 1~4 号档口中继续选择, 否则从 5~8 号档口中继续选择。以此类推, 直到只剩下一个档口。假设每个档口的美味值就是其编号, 请问小豪能吃到的美味值的期望与以下哪个值最接近? ()
- A. 5.7 B. 6.2 C. 5.0 D. 4.5
10. 倘若逆波兰表达式 (又称后缀表达式) 的值为 3, 该逆波兰表达式的部分表示是: 7 9 2 + ? 1 3 * + 4 +。请问 “?” 处应该是什么符号? ()
- A. + B. * C. / D. -
11. 若某算法的运行时间满足 $T(n) = 128T\left(\frac{n}{2^{2023}}\right) + {}^{290}\sqrt{n}$, 则 $T(n) = ()$ 。
- A. $O(n^{1/289})$ B. $O(n^{1/290})$ C. $O(n^{1/289} \log n)$ D. 无法确定

12. 班级里有 4 位学生参加了知识竞赛。知识竞赛有 4 道判断题, 4 位同学的答题情况若分别用 0 和 1 来表示选了错和对, 那么答题情况如下: 1100、1011、0011、1111。现已知这 4 人最多答对了两道题, 请你找出一组答案。()
- A. 1100 B. 0111 C. 0000 D. 1111
13. 母函数通过多项式来计算组合数学题目, 比如现在有 1 克砝码、2 克砝码、3 克砝码, 每种重量各有几种可能的获取方案? 对于 1 克砝码, 我们可以取或不取, 得到的重量贡献分别是 1 和 0, 对应的母函数是 $1 \times x^0 + 1 \times x^1 = 1 + x$, 其中 x^0 表示不取砝码产生的贡献, x^1 表示取砝码产生的贡献。对于 2 克砝码, 对应的母函数是 $1 \times x^0 + 1 \times x^2 = 1 + x^2$ 。对于 3 克砝码, 对应的母函数是 $1 \times x^0 + 1 \times x^3 = 1 + x^3$ 。将三者的母函数相乘, $(1+x)(1+x^2)(1+x^3) = 1 + x + x^2 + 2x^3 + x^4 + x^5 + x^6$, 所以可得到 7 种重量。再作深一些的解析, 三者母函数产生的 $2x^3$, 代表三者可组成两种不同情况总重量为 3 的砝码。 x^3 可以由 $1 \times 1 \times x^3$ 得到, 代表 1 克砝码、2 克砝码不取, 3 克砝码取; 也可以由 $x \times x^2 \times 1$ 得到, 代表 1 克砝码、2 克砝码取, 3 克砝码不取。问: 各位数字之和等于 11 的三位数个数是 ()。
- A. 59 B. 62 C. 60 D. 61
14. 在一个三维坐标系中有两个正方体。现已知两个正方体的所有顶点的坐标, 请问最少需要多少时间复杂度能够判断两个正方体是否相交? ()
- A. $O(n^2)$ B. $O(1)$ C. $O(n \log n)$ D. 无法判断
15. 对于下图, 假设起点为 S, 终点是 T, 从 S 到 T 的所有路径 (包含复杂路径) 的数量是 ()。



- A. 1 B. 2 C. 3 D. 4

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特别说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<iostream>
02 #include<algorithm>
03 #include<cstring>
04 #include<queue>
05
06 using namespace std;
07
08 using pii = pair<int, int>;
09 using i64 = long long;
10 using uint = unsigned int;
11 using ui64 = unsigned long long;
12
13
14 const int N = 2e6 + 10, M = N * 2;
15
16 int a[N];
17 int st[N][25], logg[N];
18
19 int getrmx(int i, int mid) {
20     int maxn = max(st[i][logg[mid - i + 1]], st[mid - (1 <<
        logg[mid - i + 1]) + 1][logg[mid - i + 1]]);
21     return maxn;
22 }
23
24 int q[N], sl[N], sr[N];
25
26 namespace IO {
27     char a; int f;
28     template<typename T> inline void read(T &x) {
29         x = 0; f = 1; a = getchar();
30         for(; ! isdigit(a); a = getchar()) if(a == '-') f = -1;
31         for(; isdigit(a); a = getchar()) x = x * 10 + a - '0';
32         x *= f;
33     }
34     template<typename T> inline void write(T x) {
35         if(x > 9) write(x / 10);
```

```
36         putchar(x % 10 + '0');
37     }
38     int read() { int x; read(x); return x; }
39     i64 read64() { i64 x; read(x); return x; }
40 }
41
42 using IO :: write;
43
44 int main(){
45     logg[1] = 0;
46     for(int i = 2; i <= 2e6; i++){
47         if((1 << (logg[i - 1] + 1)) == i){
48             logg[i] = logg[i - 1] + 1;
49         }else logg[i] = logg[i - 1];
50     }
51     memset(st,0,sizeof st);
52
53     int n;
54     cin >> n;
55     for(int i = 1; i <= n; i++) scanf("%d",&a[i]),st[i][0] = a[i];
56
57     for(int i = 1; i <= 24; i++){
58         for(int j = 1; j <= n; j++){
59             if(j + (1 << i - 1) <= n){
60                 st[j][i] = max(st[j][i - 1],st[j +
                    (1 << i - 1)][i - 1]);
61             }
62         }
63     }
64
65     int l = 1,r = 0;
66     for(int i = 1; i <= n; i++){
67         while(r >= l && a[q[r]] > a[i]) r--;
68         if(r >= l) sl[i] = q[r];
69         else sl[i] = 0;
70         q[++r] = i;
71     }
72
73     l = 1,r = 0;
74     for(int i = n; i >= 1; i--){
75         while(r >= l && a[q[r]] > a[i]) r--;
76         if(r >= l) sr[i] = q[r];
```



```
77         else sr[i] = n + 1;
78         q[++r] = i;
79     }
80
81     __int128 ans = 0;
82     for(int i = 1; i <= n; i++){
83         __int128 k = sr[i] - 1 - sl[i] - 1 + 1;
84         ans = max(ans, k * getrmax(sl[i] + 1, sr[i] - 1) * a[i]);
85     }
86
87     write(ans);
88     return 0;
89 }
```

■ 判断题

16. 若将第 51 行去掉, 程序的输出不变。 ()
17. 数组 `logg[i]` 求解的是以 2 为底 `i` 的对数。 ()
18. 当 `n` 不为 1 时, 该程序 `ans` 输出的值不可能等于 `a[1]* a[1]* n`。 ()
19. 当输入的数据为 5 3 1 2 4 5 时, `sr[3]` 的值为 5。 ()

■ 选择题

20. 当输入为 9 5 11 15 17 19 3 19 9 4 时, 输出为 ()。
- A. 855 B. 836 C. 570 D. 513
21. 当输入 10 7 3 18 9 4 5 1 7 6 5 时, `sl[5]` 的输出为 ()。
- A. 7 B. 3 C. 9 D. 2

(2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int N = 1e5 + 10;
05
06 typedef long long ll;
07 ll a[N], f[N][20];
08
09 ll ks(ll a, ll b, ll p){
10     ll res = 0;
11     while(b){
```

```
12     if(b & 1) res = (res + a) % p;
13     a = (a + a) % p;
14     b >>= 1;
15 }
16 return res;
17 }
18
19 int main(){
20     ll p;
21     int n,m;
22     scanf("%d %d %lld",&n,&m,&p);
23     for(int i = 1; i <= n; i++){
24         scanf("%lld",&a[i]);
25         f[i][0] = a[i];
26     }
27
28     for(int i = 1; i <= log2(n) + 1; i++){
29         for(int j = 1; j <= n; j++){
30             if(j + (1 << (i - 1)) <= n){
31                 f[j][i] = (f[j][i - 1] + f[j + (1 << (i - 1))][i - 1]) % p;
32             }
33         }
34     }
35
36     int l,r;
37     for(int i = 1; i <= m; i++){
38         scanf("%d %d",&l,&r);
39         int x = r - l + 1;
40         int x2 = x;
41         int d = 0; ll sum = 0;
42         while(x){
43             if(x & 1) sum += f[l][d], sum %= p, l += 1 << d;
44             d++;
45             x >>= 1;
46         }
47         printf("%lld\n",ks(sum,x2,p));
48     }
49
50 }
```

注：题目保证 $n, m \leq 10^5$ 。 $1 \leq a[i]$, $p \leq 10^{18}$ 。


```
15         if(i != pre1 && i != pre2){
16             ans += dfs(pos - 1, limit && i == up, lead && !i,
17                        (!lead || i) ? i : -1, pre1);
18         }
19         if(!limit && !lead){
20             f[pos][pre1 + 1][pre2 + 1] = ans;
21         }
22         return ans;
23     }
24
25 ll solve(ll x){
26     ll cnt = 0;
27     while(x){
28         a[++cnt] = x % 10;
29         x /= 10;
30     }
31     memset(f, -1, sizeof (f));
32     return dfs(cnt, 1, 1, -1, -1);
33 }
34 int main(){
35     cin >> l >> r;
36     cout << solve(r) - solve(l - 1);
37     return 0;
38 }
```

假设输入的 l 和 r 都是不超过 10^{18} 的自然数, 本程序的目标是统计位于区间 $[l, r]$ 中具有某种性质的正整数的数量。据此完成下列问题。

■ 判断题

28. 将第 7 行改为 `return 0;`, 程序统计出的数量不会改变。 ()
29. (3 分) 第 15 行的作用是保证对所有被统计到答案的数字中, 不会出现连续的 3 个相等的正整数。 ()
30. 当输入的 l 和 r 相等时, 输出必定为 1。 ()

■ 选择题

31. 若输入为 1 100, 则输出为 ()。
- A. 100 B. 90 C. 70 D. 80

32. (4分) 若输入为 1 1000, 则输出为 ()。

A. 991

B. 688

C. 738

D. 832

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 数组 $a[N]$ 共有 N 项, 是由 M 生成的: $a[i]=M^i \bmod 1000000007$, 其中第一项为 $a[0]$, 不是 $a[1]$, 最后一项是 $a[N-1]$ 。给定 M, N, K , 请你求出数组 $a[N]$ 中第 K 小的数。对于 100% 的数据, $2 \leq M \leq 10\,000\,000$, $1 \leq K \leq N \leq 50\,000\,000$ 。时限 1.5s, 空间限制为 131 072.0 KB。

为了减少空间的使用, 使用基数排序方法。尝试补全程序。

```
01 #include<iostream>
02 #include<cstring>
03 using namespace std;
04
05 const int N = 1e7 + 10, mod = 1e9 + 7;
06
07 int a[N], b[65540];
08
09 int main(){
10     int n, m, k;
11     cin >> m >> n >> k;
12
13     int g = ①;
14     int s = 1;
15     for(int i = 1; i <= n; i++){
16         b[②]++;
17         s = 1ll * s * m % mod;
18     }
19
20     int Hi = -1;
21     if(b[0] >= k) Hi = 0;
22     for(int i = 1; i <= 65535; i++){
23         b[i] += b[i - 1];
24         if(Hi == -1 && b[i] >= k) Hi = i, ③;
25     }
26
27     for(int i = 0; i <= 65535; i++) b[i] = 0;
28
```

```
29     s = 1;
30     for(int i = 1; i <= n; i++){
31         if(④) b[s & 65535]++;
32         s = 1ll * s * m % mod;
33     }
34
35     for(int i = 0; i <= 65535; i++){
36         if(k <= b[i]){
37             cout << ⑤ << endl;
38             break;
39         }
40         b[i + 1] += b[i];
41     }
42     return 0;
43 }
```

33. ①处应填 ()。

A. 65534

B. 65535 << 16

C. 5535 >> 16

D. 65536

34. ②处应填 ()。

A. (s & g) >> 16

B. (s & g)

C. (s & g) << 16

D. (s >> 16) & g

35. ③处应填 ()。

A. k -= b[i - 1]

B. k += b[i - 1]

C. k -= b[i]

D. k += b[i]

36. ④处应填 ()。

A. (s & g) == Hi

B. (s & g) >> 16 == Hi

C. (s >> 16) & g == Hi

D. (s & g) << 16 == Hi

37. ⑤处应填 ()。

A. Hi + i

B. Hi + i << 16

C. (Hi + i) << 16

D. Hi << 16 + i

(2) 豪豪哥是一只猫，它总是喜欢打翻家中的物品。

豪豪哥刚来到家中时是一只力量为 1、精力有 m 点的小猫。在家中，一共有 n 个物品，其中编号为 1 的物品重量为 0，编号为 2 的物品重量为 1，以此类推。若豪豪哥的力量大于或等于物品的重量，它便可以将物品打翻并且自身力量增加 1 点，但是前提是豪豪哥有足够的精力去打翻这个物品，否则豪豪哥将累死（精力为 0 时并不会累死）。注意，每个物品都会消耗豪豪哥的精力。

现在豪豪哥想知道如果它总是尽力去打翻物品（要么累死，要么打翻所有物品），那么它的能力值的期望最终是多少。由于小数点比较麻烦，因此你只需要输出期望 $\times n!$ 关于 1000000007 取模后的值就可以了。

输入格式：

第 1 行有两个数 n, m ，表示有 n 个物品和豪豪哥的初始精力（ $1 \leq n \leq 500\,000$ ， $1 \leq m \leq 10^9$ ）。接下来一行有 n 个数， a_i 表示编号为 i 的物品消耗的豪豪哥的精力（ $0 \leq a_i \leq m$ ）。

解题思路：

设 $f[i]$ 为至少打翻 i 个物品的方案数，那么恰好打翻 i 个物品的方案数为 $f[i] - f[i+1]$ ，打翻物品的期望就为 $(f[1] - f[2] + 2 * (f[2] - f[3]) + \dots + n * (f[n] - f[n+1]))/n!$ ，答案是 $(f[1] - f[2] + 2 * (f[2] - f[3]) + \dots + n * (f[n] - f[n+1]))/n!$ ，

化简即为 $\sum_{i=1}^n f[i]$ 。

假设 $\text{sum}[i]$ 为编号 $1 \sim i$ 的物品消耗的精力的总和，利用 $\text{sum}[i]$ 与 m 的比值求出 $f[i]$ 。

```
01 #include<iostream>
02 #include<cstring>
03 #include<queue>
04 #define x first
05 #define y second
06
07 using namespace std;
08
09 const int N = 5e5 + 10, mod = 1e9 + 7;
10 typedef pair<int,int> PII;
11 typedef long long ll;
12
13 ll a[N]; ll b[N], f[N];
14 ll fac[N];
15 priority_queue<PII, vector<PII>, ③ > q;
```

```
16
17 ll qpow(ll a,ll b){
18     if(b == -1) return 1;
19     ll s = 1;
20     while(b){
21         if(b & 1) s = s * a % mod;
22         a = a * a % mod;
23         b >>= 1;
24     }
25     return s;
26 }
27
28 int main(){
29     fac[0] = 1;
30     for(int i = 1; i <= 5e5; i++){
31         fac[i] = 1ll * fac[i - 1] * i % mod;
32     }
33     ll n,m;
34     cin >> n >> m;
35     for(int i = 1; i <= n; i++){
36         scanf("%lld",&a[i]);
37         b[i] = b[i - 1] + a[i];
38     }
39
40     q.push({a[1],1});
41     ll ans = 0;
42     ll sum = 0;
43     sum += 1;
44     for(int i = 1; i <= n - 1; i++){
45         q.push({a[i + 1],i + 1});
46         sum += ①;
47         sum %= mod;
48         if(②){
49             ans += qpow(2,i) * fac[n - i] % mod;
50         }else{
51             while(q.size()){
52                 PII p = q.top();
53                 if(b[i + 1] - p.x <= m) break;
54                 sum -= ④;
55                 sum %= mod;
56                 sum += mod;
```

```
57         sum %= mod;
58         q.pop();
59     }
60     ans += sum * fac[n - i] % mod;
61 }
62 ans %= mod;
63 }
64
65 if(b[n] <= m) ans += ⑤;
66
67 cout << ((ans) % mod + mod) % mod << endl;
68 return 0;
69 }
```

38. ①处应填 ()。

- | | |
|--------------|------------------|
| A. pow(2,i) | B. pow(2,i - 1) |
| C. qpow(2,i) | D. qpow(2,i - 1) |

39. ②处应填 ()。

- | | |
|--------------|------------------|
| A. b[i] <= m | B. b[i + 1] <= m |
| C. sum <= m | D. sum > m |

40. ③处应填 ()。

- | | |
|--------------|-----------------|
| A. less<int> | B. greater<int> |
| C. less<PII> | D. greater<PII> |

41. ④处应填 ()。

- | | |
|--------------------|--------------------|
| A. b[i + 1] | B. b[i] |
| C. qpow(2,p.y - 2) | D. qpow(2,p.y - 1) |

42. ⑤处应填 ()。

- | | |
|------------------|---------------------------|
| A. qpow(2,n - 1) | B. qpow(2,n) |
| C. qpow(2,n - 2) | D. qpow(2,n - 2) * fac[n] |